



Base de datos



Nest es independiente de cualquier base de datos, lo que le permite integrarse fácilmente con cualquier base de datos SQL o NoSQL. Dispone de varias opciones según sus preferencias. En general, conectar Nest a una base de datos es simplemente cargar un controlador Node.js adecuado para la base de datos, como haría con [MySQL](#) o [Fastify](#).

También puede utilizar directamente cualquier **biblioteca** de integración de base de datos Node.js de propósito general o ORM, como [TypeORM](#) (ver [TypeORM](#)), [Sequelize](#) (ver [Sequelize](#)), [Mongoose](#) (ver [Mongoose](#)), y [Prisma](#) (ver [Prisma](#)), para operar en un nivel superior de abstracción.

Para mayor comodidad, Nest ofrece una integración completa con TypeORM y Sequelize de fábrica con los paquetes [@nestjs/typeorm](#) y [@nestjs/sequelize](#) respectivamente, que abordaremos en este capítulo, y con Mongoose con [@nestjs/mongoose](#), que se aborda en [@nestjs/mongoose](#). Estas integraciones proporcionan funciones adicionales específicas de NestJS, como la inyección de modelos/repositorios, la capacidad de prueba y la configuración asíncrona, para facilitar aún más el acceso a la base de datos elegida.

Integración de TypeORM

Para la integración con bases de datos SQL y NoSQL, Nest ofrece el [@nestjs/typeorm](#) paquete TypeORM. es el mapeador relacional de objetos (ORM) más completo disponible para TypeScript. Al estar escrito en TypeScript, se integra perfectamente con el framework Nest.

Para empezar a usarlo, primero instalamos las dependencias necesarias. En este capítulo, demostraremos cómo usar el popular SGBD relacional [MySQL](#), pero TypeORM es compatible con muchas bases de datos relacionales, como PostgreSQL, Oracle, Microsoft SQL Server, SQLite e incluso bases de datos NoSQL como MongoDB. El procedimiento que explicamos en este capítulo es el mismo para cualquier base de datos compatible con TypeORM. Simplemente deberá instalar las bibliotecas de la API de cliente asociadas a la base de datos seleccionada.

```
$ npm install --save @nestjs/typeorm typeorm mysql2
```

Una vez finalizado el proceso de instalación, podemos importar el [TypeOrmModule](#) archivo [AppModule](#).

aplicación.módulo.ts

JS

```
import { Module } from '@nestjs/common';
```



```
import { TypeOrmModule } from '@nestjs/typeorm';

@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'mysql',
      host: 'localhost',
      port: 3306,
      username: 'root',
      password: 'root',
      database: 'test',
      entities: [],
      synchronize: true,
    }),
  ],
})
export class AppModule {}
```

ADVERTENCIA

Esta configuración `synchronize: true` no debe usarse en producción; de lo contrario, puede perder datos de producción.

El `forRoot()` método admite todas las propiedades de configuración expuestas por el `DataSource` constructor del paquete `typeorm`. Además, existen varias propiedades de configuración adicionales que se describen a continuación.

`retryAttempts`

Número de intentos de conexión a la base de datos (predeterminado: `10`)

`retryDelay`

Retraso entre intentos de reintento de conexión (ms) (valor predeterminado: `3000`)

`autoLoadEntities`

Si `true` , las entidades se cargarán automáticamente (predeterminado: `false`)

PISTA

Obtenga más información sobre las opciones de fuente de datos [aquí](#).

Una vez hecho esto, el TypeORM `DataSource` y `EntityManager` los objetos estarán disponibles para inyectarse en todo el proyecto (sin necesidad de importar ningún módulo), por ejemplo:

aplicación.módulo.ts

JS

```
import { DataSource } from 'typeorm';

@Module({
  imports: [TypeOrmModule.forRoot(), UsersModule],
})
export class AppModule {
  constructor(private dataSource: DataSource) {}
}
```



Patrón de repositorio

admite el **patrón de diseño de repositorios**, por lo que cada entidad tiene su propio repositorio. Estos repositorios pueden obtenerse de la fuente de datos de la base de datos.

Para continuar con el ejemplo, necesitamos al menos una entidad. Definémosla `User`.

usuario.entidad.ts

JS

```
import { Entity, Column, PrimaryGeneratedColumn } from 'typeorm';

@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  firstName: string;

  @Column()
  ...
}
```



```

    lastname: string;

    @Column({ default: true })
    isActive: boolean;
  }

```

PISTA

Obtenga más información sobre las entidades en la [documentación de TypeORM](#).

El `User` archivo de entidad se encuentra en el `users` directorio. Este directorio contiene todos los archivos relacionados con el `UsersModule`. Puedes decidir dónde guardar los archivos de tu modelo; sin embargo, recomendamos crearlos cerca de su **dominio**, en el directorio del módulo correspondiente.

Para comenzar a usar la `User` entidad, debemos informarle a TypeORM sobre ella insertándola en la `entities` matriz en las `forRoot()` opciones del método del módulo (a menos que use una ruta glob estática):

aplicación.módulo.ts

JS

```

import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { User } from '../users/user.entity';

@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'mysql',
      host: 'localhost',
      port: 3306,
      username: 'root',
      password: 'root',
      database: 'test',
      entities: [User],
      synchronize: true,
    }),
  ],
})
export class AppModule {}

```



A continuación, veamos `UsersModule` :

usuarios.módulo.ts

JS

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { UsersService } from './users.service';
import { UsersController } from './users.controller';
import { User } from './user.entity';

@Module({
  imports: [TypeOrmModule.forFeature([User])],
  providers: [UsersService],
  controllers: [UsersController],
})
export class UsersModule {}
```



Este módulo utiliza el `forFeature()` método para definir qué repositorios están registrados en el ámbito actual. Con esto en mente, podemos inyectar el método `UsersRepository` en el módulo `UsersService` mediante el `@InjectRepository()` decorador:

usuarios.servicio.ts

JS

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { User } from './user.entity';

@Injectable()
export class UsersService {
  constructor(
    @InjectRepository(User)
    private usersRepository: Repository<User>,
  ) {}

  findAll(): Promise<User[]> {
    return this.usersRepository.find();
  }
}
```



```

    }

    findOne(id: number): Promise<User | null> {
      return this.usersRepository.findOneBy({ id });
    }

    async remove(id: number): Promise<void> {
      await this.usersRepository.delete(id);
    }
  }
}

```

AVISO

No olvides importarlo `UsersModule` a la raíz `AppModule` .

Si desea usar el repositorio fuera del módulo que importa `TypeOrmModule.forFeature` , deberá reexportar los proveedores generados por este. Puede hacerlo exportando el módulo completo, como se muestra a continuación:

usuarios.módulo.ts

JS

```

import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { User } from './user.entity';

@Module({
  imports: [TypeOrmModule.forFeature([User])],
  exports: [TypeOrmModule]
})
export class UsersModule {}

```



Ahora, si importamos `UsersModule` en `UserHttpModule` , podemos utilizar `@InjectRepository(User)` en los proveedores del último módulo.

usuarios-http.module.ts

JS

```

import { Module } from '@nestjs/common';

```



```
import { UsersModule } from './users.module';
import { UsersService } from './users.service';
import { UsersController } from './users.controller';

@Module({
  imports: [UsersModule],
  providers: [UsersService],
  controllers: [UsersController]
})
export class UserHttpModule {}
```

Relaciones

Las relaciones son asociaciones establecidas entre dos o más tablas. Se basan en campos comunes de cada tabla y suelen incluir claves primarias y externas.

Hay tres tipos de relaciones:

One-to-one

Cada fila de la tabla principal tiene una sola fila asociada en la tabla externa. Utilice el `@OneToOne()` decorador para definir este tipo de relación.

One-to-many /
Many-to-one

Cada fila de la tabla principal tiene una o más filas relacionadas en la tabla externa. Utilice los decoradores `` `@OneToMany()` ` y `@ManyToOne()` `` para definir este tipo de relación.

Many-to-many

Cada fila de la tabla principal tiene muchas filas relacionadas en la tabla externa, y cada registro de la tabla externa tiene muchas filas relacionadas en la tabla principal. Utilice el `@ManyToMany()` decorador para definir este tipo de relación.

Para definir relaciones en las entidades, utilice los **decoradores** correspondientes . Por ejemplo, para definir que cada una `User` pueda tener varias fotos, utilice el `@OneToMany()` decorador.



```
import { Entity, Column, PrimaryGeneratedColumn, OneToMany } from 'typeorm';
import { Photo } from '../photos/photo.entity';

@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  firstName: string;

  @Column()
  lastName: string;

  @Column({ default: true })
  isActive: boolean;

  @OneToMany(type => Photo, photo => photo.user)
  photos: Photo[];
}
```

PISTA

Para obtener más información sobre las relaciones en TypeORM, visita la [documentación de TypeORM](#).

Cargar entidades automáticamente

Añadir entidades manualmente a la `entities` matriz de opciones de la fuente de datos puede ser tedioso. Además, hacer referencia a entidades desde el módulo raíz traspasa los límites del dominio de la aplicación y provoca la filtración de detalles de implementación a otras partes de la misma. Para solucionar este problema, se ofrece una solución alternativa. Para cargar entidades automáticamente, establezca la `autoLoadEntities` propiedad del objeto de configuración (pasado al `forRoot()` método) en `true`, como se muestra a continuación:

aplicación.módulo.ts

JS

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
```



```
@Module({
  imports: [
    TypeOrmModule.forRoot({
      ...
      autoLoadEntities: true,
    }),
  ],
})
export class AppModule {}
```

Con esa opción especificada, cada entidad registrada a través del `forFeature()` método se agregará automáticamente a la `entities` matriz del objeto de configuración.

ADVERTENCIA

Tenga en cuenta que las entidades que no se registran a través del `forFeature()` método, sino que solo se hace referencia a ellas desde la entidad (a través de una relación), no se incluirán en la `autoLoadEntities` configuración.

Definición de entidad separadora

Se puede definir una entidad y sus columnas directamente en el modelo mediante decoradores. Sin embargo, algunos usuarios prefieren definir las entidades y sus columnas en archivos separados mediante los

```
import { EntitySchema } from 'typeorm';
import { User } from './user.entity';

export const UserSchema = new EntitySchema<User>({
  name: 'User',
  target: User,
  columns: {
    id: {
      type: Number,
      primary: true,
      generated: true,
    },
    firstName: {
```



```

      type: String,
    },
    lastName: {
      type: String,
    },
    isActive: {
      type: Boolean,
      default: true,
    },
  },
  relations: {
    photos: {
      type: 'one-to-many',
      target: 'Photo', // the name of the PhotoSchema
    },
  },
});

```

ADVERTENCIA

Si proporciona la `target` opción, su `name` valor debe coincidir con el nombre de la clase de destino. Si no la proporciona, `target` puede usar cualquier nombre.

Nest te permite usar una `EntitySchema` instancia dondequiera `Entity` que se espere una, por ejemplo:

```

import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { UserSchema } from './user.schema';
import { UsersController } from './users.controller';
import { UsersService } from './users.service';

@Module({
  imports: [TypeOrmModule.forFeature([UserSchema])],
  providers: [UsersService],
  controllers: [UsersController],
})
export class UsersModule {}

```



Transacciones TypeORM

Una transacción de base de datos simboliza una unidad de trabajo realizada dentro de un sistema de gestión de bases de datos contra una base de datos, y se procesa de forma coherente y fiable, independientemente de otras transacciones. Una transacción generalmente representa cualquier cambio en una base de datos ().

Existen diversas estrategias para gestionar . Recomendamos usar la [QueryRunner](#) clase, ya que proporciona control total sobre la transacción.

Primero, necesitamos inyectar el [DataSource](#) objeto en una clase de la manera normal:

```
@Injectable()
export class UsersService {
  constructor(private dataSource: DataSource) {}
}
```



PISTA

La [DataSource](#) clase se importa desde el [typeorm](#) paquete.

Ahora, podemos usar este objeto para crear una transacción.

```
async createMany(users: User[]) {
  const queryRunner = this.dataSource.createQueryRunner();

  await queryRunner.connect();
  await queryRunner.startTransaction();
  try {
    await queryRunner.manager.save(users[0]);
    await queryRunner.manager.save(users[1]);

    await queryRunner.commitTransaction();
  } catch (err) {
    // since we have errors lets rollback the changes we made
    await queryRunner.rollbackTransaction();
  } finally {
    // you need to release a queryRunner which was manually instantiated
  }
}
```



```
    await queryRunner.release();
  }
}
```

PISTA

Tenga en cuenta que `dataSource` solo se usa para crear el objeto `QueryRunner`. Sin embargo, para probar esta clase se requeriría simular todo el `DataSource` objeto (lo que expone varios métodos). Por lo tanto, recomendamos usar una clase de fábrica auxiliar (p. ej., `QueryRunnerFactory`) y definir una interfaz con un conjunto limitado de métodos necesarios para mantener las transacciones. Esta técnica simplifica bastante la simulación de estos métodos.

Explora tu gráfico con NestJS Devtools

- ✓ Visualizador de gráficos
- ✓ Zona de juegos interactiva
- ✓ Navegador de rutas
- ✓ Integración CI/CD

Alternativamente, puede utilizar el enfoque de estilo de devolución de llamada con el `transaction` método del `DataSource` objeto ().

```
async createMany(users: User[]) {
  await this.dataSource.transaction(async manager => {
    await manager.save(users[0]);
    await manager.save(users[1]);
  });
}
```



Suscriptores

con `typeorm`, puedes escuchar eventos de entidades específicas.

```
import {
  DataSource,
  EntitySubscriberInterface,
  EventSubscriber,
  InsertEvent,
} from 'typeorm';
import { User } from '../user.entity';

@EntitySubscriber()
export class UserSubscriber implements EntitySubscriberInterface<User> {
  constructor(dataSource: DataSource) {
    dataSource.subscribers.push(this);
  }

  listenTo() {
    return User;
  }

  beforeInsert(event: InsertEvent<User>) {
    console.log('BEFORE USER INSERTED: ', event.entity);
  }
}
```



ADVERTENCIA

Los suscriptores de eventos no pueden tener **alcance de solicitud**.

Ahora, agrega la `UserSubscriber` clase a la `providers` matriz:

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { User } from '../user.entity';
import { UsersController } from '../users.controller';
import { UsersService } from '../users.service';
import { UserSubscriber } from '../user.subscriber';
```



```
@Module({
  imports: [TypeOrmModule.forFeature([User])],
  providers: [UsersService, UserSubscriber],
  controllers: [UsersController],
})
export class UsersModule {}
```

PISTA

Obtenga más información sobre los suscriptores de entidades [aquí](#).

Migraciones

permiten actualizar progresivamente el esquema de la base de datos para mantenerlo sincronizado con el modelo de datos de la aplicación, preservando al mismo tiempo los datos existentes. Para generar, ejecutar y revertir migraciones, TypeORM proporciona una `Migrator` dedicada.

Las clases de migración son independientes del código fuente de la aplicación Nest. Su ciclo de vida lo mantiene la CLI de TypeORM. Por lo tanto, no se pueden aprovechar la inyección de dependencias ni otras funciones específicas de Nest con las migraciones. Para obtener más información sobre las migraciones, consulte la guía en la [documentación](#).

Múltiples bases de datos

Algunos proyectos requieren varias conexiones a bases de datos. Esto también se puede lograr con este módulo. Para trabajar con varias conexiones, primero cree las conexiones. En este caso, es **obligatorio** nombrar la fuente de datos.

Supongamos que tiene una `Album` entidad almacenada en su propia base de datos.

```
const defaultOptions = {
  type: 'postgres',
  port: 5432,
  username: 'user',
  password: 'password',
  database: 'db',
  synchronize: true,
};
```



```
@Module({
  imports: [
    TypeOrmModule.forRoot({
      ...defaultOptions,
      host: 'user_db_host',
      entities: [User],
    }),
    TypeOrmModule.forRoot({
      ...defaultOptions,
      name: 'albumsConnection',
      host: 'album_db_host',
      entities: [Album],
    }),
  ],
})
export class AppModule {}
```

AVISO

Si no se configura `name` para una fuente de datos, su nombre se establece en `default`. Tenga en cuenta que no debe tener varias conexiones sin nombre o con el mismo nombre, ya que se sobrescribirán.

AVISO

Si usa ,

TAMBIÉN

`TypeOrmModule.forRootAsync` debe configurar el nombre de la fuente de datos fuera de `.` Por ejemplo: `useFactory`

```
TypeOrmModule.forRootAsync({
  name: 'albumsConnection',
  useFactory: ...,
  inject: ...,
}),
```



Consulte [este número](#) para obtener más detalles.

debe indicar al `TypeOrmModule.forFeature()` método y al `@InjectRepository()` decorador qué fuente de datos debe usarse. Si no proporciona ningún nombre de fuente de datos, `default` se usará esta.

```
@Module({
  imports: [
    TypeOrmModule.forFeature([User]),
    TypeOrmModule.forFeature([Album], 'albumsConnection'),
  ],
})
export class AppModule {}
```



También puedes inyectar el `DataSource` o `EntityManager` para una fuente de datos determinada:

```
@Injectable()
export class AlbumsService {
  constructor(
    @InjectDataSource('albumsConnection')
    private dataSource: DataSource,
    @InjectEntityManager('albumsConnection')
    private entityManager: EntityManager,
  ) {}
}
```



También es posible inyectar cualquiera `DataSource` a los proveedores:

```
@Module({
  providers: [
    {
      provide: AlbumsService,
      useFactory: (albumsConnection: DataSource) => {
        return new AlbumsService(albumsConnection);
      },
      inject: [getDataSourceToken('albumsConnection')],
    },
  ],
})
```



```
  })
  export class AlbumsModule {}
```

Prueba

Al realizar pruebas unitarias en una aplicación, solemos evitar la conexión a la base de datos, manteniendo la independencia de nuestras suites de pruebas y su ejecución lo más rápida posible. Sin embargo, nuestras clases podrían depender de repositorios extraídos de la instancia de la fuente de datos (conexión). ¿Cómo lo gestionamos? La solución es crear repositorios simulados. Para ello, configuramos

Cada repositorio registrado se representa automáticamente mediante un `<EntityName>Repository` token, donde `EntityName` <nombre de la clase de entidad> es el nombre de la clase de entidad.

El `@nestjs/typeorm` paquete expone la `getRepositoryToken()` función que devuelve un token preparado en función de una entidad determinada.

```
@Module({
  providers: [
    UsersService,
    {
      provide: getRepositoryToken(User),
      useValue: mockRepository,
    },
  ],
})
export class UsersModule {}
```



`mockRepository` Ahora se usará un sustituto como `UsersRepository`. Siempre que una clase solicite `UsersRepository` el uso de un `@InjectRepository()` decorador, Nest usará el `mockRepository` objeto registrado.

Configuración asíncrona

Quizás prefiera pasar las opciones del módulo del repositorio de forma asíncrona en lugar de estática. En este caso, utilice el `forRootAsync()` método, que ofrece varias maneras de gestionar la configuración asíncrona.

Un enfoque es utilizar una función de fábrica:

```
TypeOrmModule.forRootAsync({
```



```
useFactory: () => ({
  type: 'mysql',
  host: 'localhost',
  port: 3306,
  username: 'root',
  password: 'root',
  database: 'test',
  entities: [],
  synchronize: true,
}),
});
```

Nuestra fábrica se comporta como cualquier otro de inyectar dependencias a través de `inject`).

(por ejemplo, puede ser `async` y es capaz

```
TypeOrmModule.forRootAsync({
  imports: [ConfigModule],
  useFactory: (configService: ConfigService) => ({
    type: 'mysql',
    host: configService.get('HOST'),
    port: +configService.get('PORT'),
    username: configService.get('USERNAME'),
    password: configService.get('PASSWORD'),
    database: configService.get('DATABASE'),
    entities: [],
    synchronize: true,
  }),
  inject: [ConfigService],
});
```



Alternativamente, puede utilizar la `useClass` sintaxis:

```
TypeOrmModule.forRootAsync({
  useClass: TypeOrmConfigService,
});
```



La construcción anterior instanciará `TypeOrmConfigService` dentro `TypeOrmModule` y la usará para proporcionar un objeto de opciones mediante una llamada a `createTypeOrmOptions()`. Tenga en cuenta que esto significa que `TypeOrmConfigService` debe implementar la `TypeOrmOptionsFactory` interfaz, como se muestra a continuación:

```
@Injectable()
export class TypeOrmConfigService implements TypeOrmOptionsFactory {
  createTypeOrmOptions(): TypeOrmModuleOptions {
    return {
      type: 'mysql',
      host: 'localhost',
      port: 3306,
      username: 'root',
      password: 'root',
      database: 'test',
      entities: [],
      synchronize: true,
    };
  }
}
```



Para evitar la creación `TypeOrmConfigService` interna `TypeOrmModule` y el uso de un proveedor importado de un módulo diferente, puede utilizar la `useExisting` sintaxis.

```
TypeOrmModule.forRootAsync({
  imports: [ConfigModule],
  useExisting: ConfigService,
});
```



Esta construcción funciona de la misma manera `useClass` con una diferencia crítica: `TypeOrmModule` buscará módulos importados para reutilizar uno existente `ConfigService` en lugar de crear una instancia de uno nuevo.

PISTA

Asegúrese de que la `name` propiedad esté definida al mismo nivel que la propiedad `useFactory`, `useClass`, o `useValue`. Esto permitirá que Nest registre correctamente la fuente de datos con el token de inyección correspondiente.

Fábrica de fuentes de datos personalizadas

Junto con la configuración asíncrona usando `useFactory` , `useClass` , o `useExisting` , puede especificar opcionalmente una `dataSourceFactory` función que le permitirá proporcionar su propia fuente de datos TypeORM en lugar de permitirle `TypeOrmModule` crear la fuente de datos.

`dataSourceFactory` recibe el TypeORM `DataSourceOptions` configurado durante la configuración asíncrona usando `useFactory` , `useClass` , o `useExisting` y devuelve un `Promise` que resuelve un TypeORM `DataSource` .

```
TypeOrmModule.forRootAsync({
  imports: [ConfigModule],
  inject: [ConfigService],
  // Use useFactory, useClass, or useExisting
  // to configure the DataSourceOptions.
  useFactory: (configService: ConfigService) => ({
    type: 'mysql',
    host: configService.get('HOST'),
    port: +configService.get('PORT'),
    username: configService.get('USERNAME'),
    password: configService.get('PASSWORD'),
    database: configService.get('DATABASE'),
    entities: [],
    synchronize: true,
  }),
  // dataSource receives the configured DataSourceOptions
  // and returns a Promise<DataSource>.
  dataSourceFactory: async (options) => {
    const dataSource = await new DataSource(options).initialize();
    return dataSource;
  },
});
```



PISTA

La `DataSource` clase se importa desde el `typeorm` paquete.

Ejemplo

Un ejemplo práctico está disponible [aquí](#).

Soporte empresarial oficial

- ✓ Proporcionar orientación técnica
- ✓ Realizar revisiones de código en profundidad
- ✓ Miembros del equipo de mentoría
- ✓ Asesoramiento sobre las mejores prácticas

Integración de Sequelize

Una alternativa a usar TypeORM es usar el ORM [Sequelize](#) con el [@nestjs/sequelize](#) paquete. Además, utilizamos el paquete [sequelize-typescript](#), que proporciona un conjunto de decoradores adicionales para definir entidades declarativamente.

Para empezar a usarlo, primero instalamos las dependencias necesarias. En este capítulo, demostraremos cómo usar el popular SGBD relacional [MySQL](#), pero Sequelize es compatible con muchas bases de datos relacionales, como PostgreSQL, MySQL, Microsoft SQL Server, SQLite y MariaDB. El procedimiento que explicamos en este capítulo es el mismo para cualquier base de datos compatible con Sequelize. Simplemente deberá instalar las bibliotecas de la API de cliente asociadas a la base de datos seleccionada.

```
$ npm install --save @nestjs/sequelize sequelize sequelize-typescript mysql2
$ npm install --save-dev @types/sequelize
```

Una vez finalizado el proceso de instalación, podemos importar el [SequelizeModule](#) archivo [AppModule](#).

aplicación.módulo.ts

JS

```
import { Module } from '@nestjs/common';
import { SequelizeModule } from '@nestjs/sequelize';
```



```
import { SequelizeModule } from '@nestjs/sequelize';

@Module({
  imports: [
    SequelizeModule.forRoot({
      dialect: 'mysql',
      host: 'localhost',
      port: 3306,
      username: 'root',
      password: 'root',
      database: 'test',
      models: [],
    }),
  ],
})
export class AppModule {}
```

El `forRoot()` método admite todas las propiedades de configuración expuestas por el constructor Sequelize (). Además, se describen a continuación varias propiedades de configuración adicionales.

`retryAttempts`

Número de intentos de conexión a la base de datos (predeterminado: `10`)

`retryDelay`

Retraso entre intentos de reintento de conexión (ms) (valor predeterminado: `3000`)

`autoLoadModels`

Si `true` , los modelos se cargarán automáticamente (predeterminado: `false`)

`keepConnectionAlive`

Si `true` , la conexión no se cerrará al apagar la aplicación (predeterminado: `false`)

`synchronize`

Si `true` , los modelos cargados automáticamente se sincronizarán (predeterminado: `true`)

Una vez hecho esto, el [Sequelize](#) objeto estará disponible para inyectarse en todo el proyecto (sin necesidad de importar ningún módulo), por ejemplo:

aplicación.servicio.ts

JS

```
import { Injectable } from '@nestjs/common';
import { Sequelize } from 'sequelize-typescript';

@Injectable()
export class AppService {
  constructor(private sequelize: Sequelize) {}
}
```



Modelos

Sequelize implementa el patrón Registro Activo. Con este patrón, se utilizan clases de modelo directamente para interactuar con la base de datos. Para continuar con el ejemplo, necesitamos al menos un modelo. Definémoslo [User](#) .

usuario.modelo.ts

JS

```
import { Column, Model, Table } from 'sequelize-typescript';

@Table
export class User extends Model {
  @Column
  firstName: string;

  @Column
  lastName: string;

  @Column({ defaultValue: true })
  isActive: boolean;
}
```



PISTA

Obtenga más información sobre los decoradores disponibles [aquí](#)

El `User` archivo del modelo se encuentra en el `users` directorio. Este directorio contiene todos los archivos relacionados con el módulo `UsersModule`. Puedes decidir dónde guardar los archivos del modelo; sin embargo, recomendamos crearlos cerca de su **dominio**, en el directorio del módulo correspondiente.

Para comenzar a utilizar el `User` modelo, debemos informar a Sequelize sobre él insertándolo en la `models` matriz en las `forRoot()` opciones del método del módulo:

aplicación.módulo.ts

JS

```
import { Module } from '@nestjs/common';
import { SequelizeModule } from '@nestjs/sequelize';
import { User } from '../users/user.model';

@Module({
  imports: [
    SequelizeModule.forRoot({
      dialect: 'mysql',
      host: 'localhost',
      port: 3306,
      username: 'root',
      password: 'root',
      database: 'test',
      models: [User],
    }),
  ],
})
export class AppModule {}
```



A continuación, veamos `UsersModule` :

usuarios.módulo.ts

JS

```
import { Module } from '@nestjs/common';
import { SequelizeModule } from '@nestjs/sequelize';
import { User } from './user.model';
import { UsersController } from './users.controller';
```



```
import { UsersService } from './users.service';

@Module({
  imports: [SequelizeModule.forFeature([User])],
  providers: [UsersService],
  controllers: [UsersController],
})
export class UsersModule {}
```

Este módulo utiliza el `forFeature()` método para definir qué modelos se registran en el ámbito actual. Con esto en mente, podemos inyectar el método `UserModel` en `UsersService` el `@InjectModel()` decorador:

usuarios.servicio.ts

JS

```
import { Injectable } from '@nestjs/common';
import { InjectModel } from '@nestjs/sequelize';
import { User } from './user.model';

@Injectable()
export class UsersService {
  constructor(
    @InjectModel(User)
    private userModel: typeof User,
  ) {}

  async findAll(): Promise<User[]> {
    return this.userModel.findAll();
  }

  findOne(id: string): Promise<User> {
    return this.userModel.findOne({
      where: {
        id,
      },
    });
  }

  async remove(id: string): Promise<void> {
    const user = await this.findOne(id);
```



```
    await user.destroy();  
  }  
}
```

AVISO

No olvides importarlo `UsersModule` a la raíz `AppModule` .

Si desea usar el modelo fuera del módulo que importa `SequelizeModule.forFeature` , deberá reexportar los proveedores generados por este. Puede hacerlo exportando el módulo completo, como se muestra a continuación:

usuarios.módulo.ts

JS

```
import { Module } from '@nestjs/common';  
import { SequelizeModule } from '@nestjs/sequelize';  
import { User } from './user.entity';  
  
@Module({  
  imports: [SequelizeModule.forFeature([User])],  
  exports: [SequelizeModule]  
})  
export class UsersModule {}
```



Ahora, si importamos `UsersModule` en `UserHttpModule` , podemos utilizar `@InjectModel(User)` en los proveedores del último módulo.

usuarios-http.module.ts

JS

```
import { Module } from '@nestjs/common';  
import { UsersModule } from './users.module';  
import { UsersService } from './users.service';  
import { UsersController } from './users.controller';  
  
@Module({  
  imports: [UsersModule],  
  providers: [UsersService],  
})
```



```

    controllers: [UsersController]
  })
  export class UserHttpModule {}

```

Relaciones

Las relaciones son asociaciones establecidas entre dos o más tablas. Se basan en campos comunes de cada tabla y suelen incluir claves primarias y externas.

Hay tres tipos de relaciones:

One-to-one

Cada fila de la tabla principal tiene una y sólo una fila asociada en la tabla externa

One-to-many /

Many-to-one

Cada fila de la tabla principal tiene una o más filas relacionadas en la tabla externa

Many-to-many

Cada fila de la tabla principal tiene muchas filas relacionadas en la tabla externa, y cada registro de la tabla externa tiene muchas filas relacionadas en la tabla principal.

Para definir relaciones en los modelos, utilice los **decoradores** correspondientes . Por ejemplo, para definir que cada uno `User` pueda tener varias fotos, utilice el `@HasMany()` decorador.

usuario.modelo.ts

JS

```

import { Column, Model, Table, HasMany } from 'sequelize-typescript';
import { Photo } from '../photos/photo.model';

```



```

@Table
export class User extends Model {
  @Column
  firstName: string;

  @Column
  lastName: string;

```

```
@Column({ defaultValue: true })
isActive: boolean;

@HasMany(() => Photo)
photos: Photo[];
}
```

PISTA

Para obtener más información sobre las asociaciones en Sequelize, lea [este](#) capítulo.

Modelos de carga automática

Añadir modelos manualmente a la `models` matriz de opciones de conexión puede ser tedioso. Además, hacer referencia a modelos desde el módulo raíz traspasa los límites del dominio de la aplicación y provoca la filtración de detalles de implementación a otras partes de la aplicación. Para solucionar este problema, cargue los modelos automáticamente estableciendo las propiedades `autoLoadModels` y `synchronize` del objeto de configuración (pasado al `forRoot()` método) en `true`, como se muestra a continuación:

aplicación.módulo.ts

JS

```
import { Module } from '@nestjs/common';
import { SequelizeModule } from '@nestjs/sequelize';

@Module({
  imports: [
    SequelizeModule.forRoot({
      ...
      autoLoadModels: true,
      synchronize: true,
    }),
  ],
})
export class AppModule {}
```



Con esa opción especificada, cada modelo registrado a través del `forFeature()` método se agregará

automáticamente a la `models` matriz del objeto de configuración.

ADVERTENCIA

Tenga en cuenta que los modelos que no se registran a través del `forFeature()` método, sino que solo se referencian desde el modelo (a través de una asociación), no se incluirán.

Secuenciar transacciones

Una transacción de base de datos simboliza una unidad de trabajo realizada dentro de un sistema de gestión de bases de datos contra una base de datos, y se procesa de forma coherente y fiable, independientemente de otras transacciones. Una transacción generalmente representa cualquier cambio en una base de datos ().

Existen diversas estrategias para gestionar . A continuación, se muestra un ejemplo de implementación de una transacción gestionada (auto-callback).

Primero, necesitamos inyectar el `Sequelize` objeto en una clase de la manera normal:

```
@Injectable()
export class UsersService {
  constructor(private sequelize: Sequelize) {}
}
```



PISTA

La `Sequelize` clase se importa desde el `sequelize-typescript` paquete.

Ahora, podemos usar este objeto para crear una transacción.

```
async createMany() {
  try {
    await this.sequelize.transaction(async t => {
      const transactionHost = { transaction: t };

      await this.userModel.create(
        { firstName: 'Abraham', lastName: 'Lincoln' },

```



```
        transactionHost,
      );
      await this.userModel.create(
        { firstName: 'John', lastName: 'Boothe' },
        transactionHost,
      );
    });
  } catch (err) {
    // Transaction has been rolled back
    // err is whatever rejected the promise chain returned to the transaction callback
  }
}
```

PISTA

Tenga en cuenta que la [Sequelize](#) instancia se usa solo para iniciar la transacción. Sin embargo, para probar esta clase se requeriría simular todo el [Sequelize](#) objeto (lo que expone varios métodos). Por lo tanto, recomendamos usar una clase de fábrica auxiliar (p. ej., [TransactionRunner](#)) y definir una interfaz con un conjunto limitado de métodos necesarios para mantener las transacciones. Esta técnica simplifica bastante la simulación de estos métodos.

Migraciones

permiten actualizar progresivamente el esquema de la base de datos para mantenerlo sincronizado con el modelo de datos de la aplicación, conservando al mismo tiempo los datos existentes. Para generar, ejecutar y revertir migraciones, Sequelize proporciona una [CLI](#) dedicada.

Las clases de migración son independientes del código fuente de la aplicación Nest. Su ciclo de vida lo mantiene la CLI de Sequelize. Por lo tanto, no se pueden aprovechar la inyección de dependencias ni otras funciones específicas de Nest con las migraciones. Para obtener más información sobre las migraciones, consulte la guía de la

¡Aprende de la manera **correcta** !

- ✓ más de 80 capítulos
- ✓ Sesiones de inmersión
- ✓ Más de 5 horas de vídeos
- profunda

Múltiples bases de datos

Algunos proyectos requieren varias conexiones a bases de datos. Esto también se puede lograr con este módulo. Para trabajar con varias conexiones, primero cree las conexiones. En este caso, es **obligatorio** nombrar las conexiones.

Supongamos que tiene una `Album` entidad almacenada en su propia base de datos.

```
const defaultOptions = {
  dialect: 'postgres',
  port: 5432,
  username: 'user',
  password: 'password',
  database: 'db',
  synchronize: true,
};

@Module({
  imports: [
    SequelizeModule.forRoot({
      ...defaultOptions,
      host: 'user_db_host',
      models: [User],
    }),
    SequelizeModule.forRoot({
      ...defaultOptions,
      name: 'albumsConnection',
      host: 'album_db_host',
      models: [Album],
    }),
  ],
})
export class AppModule {}
```



AVISO

Si no se configura `name` para una conexión, su nombre se establece en `default`. Tenga en cuenta que no debe tener varias conexiones sin nombre o con el mismo nombre, ya que se sobrescribirán.

En este punto, ya tiene `User` los `Album` modelos registrados con su propia conexión. Con esta configuración, debe indicar al `SequelizeModule.forFeature()` método y al `@InjectModel()` decorador qué conexión debe usarse. Si no proporciona ningún nombre de conexión, `default` se usará.

```
@Module({
  imports: [
    SequelizeModule.forFeature([User]),
    SequelizeModule.forFeature([Album], 'albumsConnection'),
  ],
})
export class AppModule {}
```



También puedes inyectar la `Sequelize` instancia para una conexión determinada:

```
@Injectable()
export class AlbumsService {
  constructor(
    @InjectConnection('albumsConnection')
    private sequelize: Sequelize,
  ) {}
}
```



También es posible inyectar cualquier `Sequelize` instancia a los proveedores:

```
@Module({
  providers: [
    {
      provide: AlbumsService,
      useFactory: (albumsSequelize: Sequelize) => {
```



```

    return new AlbumsService(albumsSequelize);
  },
  inject: [getDataSourceToken('albumsConnection')],
},
],
}))
export class AlbumsModule {}

```

Prueba

Al realizar pruebas unitarias en una aplicación, solemos evitar una conexión a la base de datos, manteniendo la independencia de nuestras suites de pruebas y su proceso de ejecución lo más rápido posible. Sin embargo, nuestras clases podrían depender de modelos extraídos de la instancia de conexión. ¿Cómo lo gestionamos? La solución es crear modelos simulados. Para ello, configuramos `MockedModels`. Cada modelo registrado se representa automáticamente mediante un `<ModelName>Model` token, donde `ModelName` <nombre de la clase del modelo> es el nombre de la clase del modelo.

El `@nestjs/sequelize` paquete expone la `getModelToken()` función que devuelve un token preparado basado en un modelo determinado.

```

@Module({
  providers: [
    UsersService,
    {
      provide: getModelToken(User),
      useValue: mockModel,
    },
  ],
})
export class UsersModule {}

```



`mockModel` Ahora se usará un sustituto como `UserModel`. Siempre que una clase solicite `UserModel` el uso de un `@InjectModel()` decorador, Nest usará el `mockModel` objeto registrado.

Configuración asíncrona

Quizás prefiera pasar sus `SequelizeModule` opciones de forma asíncrona en lugar de estática. En este caso, utilice el `forRootAsync()` método, que ofrece varias maneras de gestionar la configuración asíncrona.

Un enfoque es utilizar una función de fábrica:

```
SequelizeModule.forRootAsync({  
  useFactory: () => ({  
    dialect: 'mysql',  
    host: 'localhost',  
    port: 3306,  
    username: 'root',  
    password: 'root',  
    database: 'test',  
    models: [],  
  }),  
});
```



Nuestra fábrica se comporta como cualquier otro de inyectar dependencias a través de `inject`).

(por ejemplo, puede ser `async` y es capaz

```
SequelizeModule.forRootAsync({  
  imports: [ConfigModule],  
  useFactory: (configService: ConfigService) => ({  
    dialect: 'mysql',  
    host: configService.get('HOST'),  
    port: +configService.get('PORT'),  
    username: configService.get('USERNAME'),  
    password: configService.get('PASSWORD'),  
    database: configService.get('DATABASE'),  
    models: [],  
  }),  
  inject: [ConfigService],  
});
```



Alternativamente, puede utilizar la `useClass` sintaxis:

```
SequelizeModule.forRootAsync({  
  useClass: SequelizeConfigService,  
  ...
```



```
});
```

La construcción anterior instanciará `SequelizeConfigService` dentro `SequelizeModule` y la usará para proporcionar un objeto de opciones mediante una llamada a `createSequelizeOptions()`. Tenga en cuenta que esto significa que `SequelizeConfigService` debe implementar la `SequelizeOptionsFactory` interfaz, como se muestra a continuación:

```
@Injectable()
class SequelizeConfigService implements SequelizeOptionsFactory {
  createSequelizeOptions(): SequelizeModuleOptions {
    return {
      dialect: 'mysql',
      host: 'localhost',
      port: 3306,
      username: 'root',
      password: 'root',
      database: 'test',
      models: [],
    };
  }
}
```



Para evitar la creación `SequelizeConfigService` interna `SequelizeModule` y el uso de un proveedor importado de un módulo diferente, puede utilizar la `useExisting` sintaxis.

```
SequelizeModule.forRootAsync({
  imports: [ConfigModule],
  useExisting: ConfigService,
});
```



Esta construcción funciona de la misma manera `useClass` con una diferencia crítica: `SequelizeModule` buscará módulos importados para reutilizar uno existente `ConfigService` en lugar de crear una instancia de uno nuevo.

Ejemplo

Un ejemplo práctico está disponible [aquí](#).

Apóyanos

Nest es un proyecto de código abierto con licencia del MIT. Puede crecer gracias al apoyo de estas personas excepcionales. Si quieres unirte a ellos, lee más [aquí](#).

Patrocinadores principales

Patrocinadores / Socios



TRILON.



Suscríbete a nuestro boletín informativo

¡Suscríbete para mantenerte actualizado con las últimas actualizaciones, funciones y videos de Nest!

Dirección de correo electrónico..

Copyright © 2017-2025 MIT por
Consultoría oficial de NestJS

Diseño de
alojado por

